NumPy Functions — Complete Reference

Numerical Python | Scientific Computing Library

1. What is NumPy?

NumPy (Numerical Python) is an open-source Python library used for scientific and numerical computation. It provides a powerful N-dimensional array object (ndarray) that supports fast mathematical operations.

Import Statement: import numpy as np

Key Difference: Python List vs NumPy Array

Feature	Python List	NumPy Array
Data Types	Heterogeneous (mixed types allowed)	Homogeneous (same type only)
Multiplication	Repeats the list	Multiplies each element (element-wise)
Speed	Slower for math	Much faster for math
Memory	Uses more memory	Uses less memory

---

2. np.array()

Converts a Python list (or nested list) into a NumPy ndarray (N-Dimensional Array). This is the most fundamental function in NumPy.

Syntax: np.array(object)
Parameter: object — A Python list, tuple, or nested list
Returns: numpy.ndarray

Examples:

```
a = [1, 2, 3]
b = np.array(a)       # → array([1, 2, 3])
```

```
# 2D array (matrix)
a = [[1,2,3],[4,5,6],[7,8,9]]
b = np.array(a)
# → array([[1, 2, 3],
#          [4, 5, 6],
#          [7, 8, 9]])
```

Why use it?

- Enables mathematical operations on entire arrays at once
- a * 3 on a list repeats it; on a NumPy array it multiplies each element by 3
- Type becomes numpy.ndarray instead of list

---

3. Array Properties

Once you create a NumPy array, these three properties help you understand its structure.

3.1 .shape

Returns a tuple showing the number of rows and columns (dimensions) of the array.

Syntax: array.shape

Returns: Tuple — (rows, columns) for 2D; (n,) for 1D

```
b = np.array([[1,2,3],[4,5,6],[7,8,9]])
b.shape      # → (3, 3)   means 3 rows, 3 columns
```

3.2 .size

Returns the total number of elements in the array. It is calculated as rows × columns.

Syntax: array.size

Returns: Integer — total count of all elements

```
b.size       # → 9   (3 rows × 3 columns = 9 elements)
```

3.3 .ndim

Returns the number of dimensions (axes) of the array.

Syntax: array.ndim

Returns: Integer — 1 for 1D array, 2 for 2D matrix, 3 for 3D, etc.

```
b.ndim       # → 2   (it's a 2-dimensional matrix)
```

Quick Reference Table:

Property	What it tells you	Example Output
.shape	Rows and columns of the array	(3, 3)

.size	Total number of elements	9
.ndim	Number of dimensions	2

---

4. np.zeros()

Creates a NumPy array where ALL values are initialized to 0.0 (floating point). This is commonly used to create empty arrays before filling them with actual data.

Syntax: np.zeros(shape)
Parameter: shape — An integer (for 1D) or a tuple (rows, columns) for 2D
Default dtype: float64 (values are 0.0, not 0)
Returns: numpy.ndarray filled with zeros

Examples:

```
# 1D array with 3 zeros
np.zeros(3)         # → array([0., 0., 0.])
```

```
# 2D array with 3 rows and 4 columns
np.zeros((3, 4))
# → array([[0., 0., 0., 0.],
#          [0., 0., 0., 0.],
#          [0., 0., 0., 0.]])
```

When to use:

- Initialize weight matrices in machine learning
- Create placeholder arrays before filling with computed values
- Reset or clear an existing array's content

---

5. np.ones()

Creates a NumPy array where ALL values are initialized to 1.0 (floating point). It works exactly like np.zeros() but fills with 1 instead of 0.

Syntax: np.ones(shape)
Parameter: shape — An integer (for 1D) or a tuple (rows, columns) for 2D
Default dtype: float64 (values are 1.0, not 1)
Returns: numpy.ndarray filled with ones

Examples:

```
# 1D array with 4 ones
np.ones(4)          # → array([1., 1., 1., 1.])
```

```
# 2D array with 2 rows and 3 columns
np.ones((2, 3))
# → array([[1., 1., 1.],
#          [1., 1., 1.]])
```

```
# 5x6 array example
a = np.ones((5, 6))
a.shape    # → (5, 6)
a.ndim     # → 2
a.size     # → 30
```

When to use:

- Initialize bias terms in neural networks
- Create identity-like arrays before scaling
- Use as a multiplier array to broadcast values

---

6. Complete Functions Summary

Function / Property	Purpose	Example	Output
np.array(list)	Convert list to NumPy array	np.array([1,2,3])	array([1, 2, 3])
.shape	Get rows and columns	b.shape	(3, 3)
.size	Total elements count	b.size	9
.ndim	Number of dimensions	b.ndim	2
np.zeros(n)	1D array of zeros	np.zeros(3)	array([0., 0., 0.])
np.zeros((r,c))	2D array of zeros	np.zeros((3,4))	3x4 array of 0.0
np.ones(n)	1D array of ones	np.ones(4)	array([1., 1., 1., 1.])
np.ones((r,c))	2D array of ones	np.ones((2,3))	2x3 array of 1.0